Classification ML Study Guide

Chapter 1: Classification

A **classification** supervised machine learning problem involves predicting a categorical label or class for a given input based on past observations. This is based on predicting a **probability** for each class and choosing the label with the highest predicted probability. All probabilities should add up to 1.

E.g 40% chance of default vs 60% chance of non-default -> Predicting “non-default”

Examples of Classification Problems:

- **Binary Classification**: Two possible classes (e.g., email spam detection: spam vs. not spam).
- **Multi-class Classification**: More than two possible classes (e.g., handwritten digit recognition: digits 0-9, Wine: red, white, rose).
- **Multi-label Classification**: Each input can belong to multiple classes simultaneously (e.g., tagging images with multiple objects like "cat," "dog," and "car").

Binary classification is the most common.

https://www.datacamp.com/blog/classification-machine-learning

Chapter 2: Algorithms

Support Vector Machines (SVM) Overview:

SVM is a supervised learning algorithm that can be used for both classification and regression tasks. The primary goal of SVM is to find a *hyperplane* in an N-dimensional space (N — the number of features) that distinctly separates the data points.

If the data cannot be separated by a straight line (or hyperplane in higher dimensions), SVM uses the **kernel trick** to transform the data into a higher-dimensional space where a hyperplane can be found

Key Advantages of SVM:

- **Effective in high-dimensional spaces**: SVM is particularly well-suited for problems where the number of features is large compared to the number of data points.

- **Robust to Overfitting**: SVM aims to maximize the margin, which tends to reduce the risk of overfitting, especially in high-dimensional spaces.
- **Versatile**: Through the use of kernels, SVM can handle *both linear and non-linear* data.

Key Disadvantages of SVM:

- **Computationally Intensive**: Training an SVM can be computationally expensive, especially with large datasets.
- **Choice of Kernel**: The performance of SVM is highly dependent on the choice of the kernel and its parameters, which may require extensive experimentation.
- **Lack of Explainability:** Even though this is a MATH based model, it is still hard to explain why the model is making the predictions.

TL;DR:

- Support Vector Machines are MATH based, complex algorithms that are used for both regression & classification problems (although they are more utilized for classification)
- They project your feature space into *higher dimensions* and use *hyperplanes* to find patterns in the data.
- Pros:
    - Math based, so only one answer and technically has an equation
    - Can capture non-linear effects that Logistic Regressions struggle with
- Cons:
    - Very complicated and difficult to explain
    - High computation cost
    - Only really effective with a high-dimension feature set (dozens of features)
    - Are typically out-performed by Trees anyway

K-Nearest Neighbors (KNN) Overview:

K-Nearest Neighbors (KNN) is a simple yet powerful algorithm used in machine learning for both classification, regression, and even recommendation systems. KNN

For each data point, the distance is calculated between it and all the points in the training set. Common distance metrics include *Euclidean distance, Manhattan distance, and Cosine Similarity.* Distance Metrics

- In KNN classification, the algorithm assigns a class to a data point based on the majority class among its K nearest neighbors in the training set.
- In KNN regression, the algorithm predicts a continuous value for a data point by averaging the values of its K nearest neighbors.

- In recommendation systems, KNN can be used to find similar items or users based on their preferences or features. The idea is to recommend items that are "close" to what a user likes or what similar users have liked.

TL:DR:

- KNN is a simple, MATH based model. I call it the “common sense” model. Find other points in the training set similar to your input and use the average/mode of those similar points to generate a prediction.
- Calculating distance matters. Euclidean, Manhattan, and Cosine Similarity are all ways to measure distance between points.
- Pros:
    - Simple and Explainable
    - Can find complex relationships/non-linear data
- Cons:
    - Doesn’t work well on large-scale or high-dimensional datasets
    - Exponential computational cost on big data
    - Memory Intensive
        - KNN requires storing the entire training dataset since it relies on it to make predictions. This can be problematic when working with large datasets, especially in memory-constrained environments.
    - Highly dependent on choice of K
    - Typically out-performed by Trees

KNN is best suited for small to medium-sized datasets where interpretability and simplicity are priorities. It works well when the dataset has a relatively low number of dimensions and when the features are properly scaled and relevant to the task.

KNN may not be ideal for large-scale or high-dimensional datasets due to its computational cost and memory usage.

Logistic Regression

Logistic Regression is a *linear model* classification algorithm used to predict the probability of a categorical outcome. Despite its name, logistic regression is not used for regression tasks but for classification. It is similar to linear regression but applies a **sigmoid function** to map the output to a probability between 0 and 1, producing an **S-curve**.

The model is trained (and proven) using **Maximum Likelihood Estimation (MLE)**, unlike linear regression, which typically uses the **Least Squares** method. **Regularization** (L1 and L2) can be applied to prevent overfitting by penalizing large coefficients in the model.

Logistic regression is widely used due to its simplicity, interpretability, and effectiveness in binary classification problems. It is efficient, works well with large datasets, and can be regularized to prevent overfitting.

However, it is limited by its linear nature, sensitivity to outliers, and assumptions about feature independence. For complex, non-linear relationships, or when dealing with highly correlated features, other models like decision trees, SVMs, or neural networks might be more appropriate.

TL;DR

Pros:

- Linear Model
    - Equation, simple, explainable
    - S-Curve is easy to relate to probabilities (like line of best fit)
    - Proven in mathematics

Cons:

- Needs linear data, can't really capture non-linearity
- Doesn't play with with multicollinearity
- Suffers from assumptions of linear models

Logistic Regression is the baseline linear model for classification problems.

Chapter 3: Evaluation

Section 1: Type 1/Type 2 Errors, Confusion Matrix, True/False Positives/Negatives

True/False Positives/Negatives are the root of evaluating any classification model performance, just like the residual is the root of evaluating regression model performance.

Confusion Matrix:

A **confusion matrix** is a table used to evaluate the performance of a classification model. It compares the actual target values with those predicted by the model and helps visualize the types of errors made. For binary classification, the confusion matrix is a 2x2 table:

	Predicted Negative	**Predicted Positive**

Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- **True Positive (TP)**: The model correctly predicts the positive class (e.g., predicting a patient has a disease when they actually do).
- **False Positive (FP)**: The model incorrectly predicts the positive class (e.g., predicting a patient has a disease when they don't). Also known as a **Type 1 Error**.
- **True Negative (TN)**: The model correctly predicts the negative class (e.g., predicting a patient does not have a disease when they actually don't).
- **False Negative (FN)**: The model incorrectly predicts the negative class (e.g., predicting a patient does not have a disease when they actually do). Also known as a **Type 2 Error**.

Examples:

- **Spam Detection:**
  - **TP**: An email correctly identified as spam.
  - **FP**: A non-spam email incorrectly classified as spam.
  - **TN**: A non-spam email correctly identified as non-spam.
  - **FN**: A spam email incorrectly classified as non-spam.
- **Medical Diagnosis:**
  - **TP**: A patient with the disease correctly diagnosed as having the disease.
  - **FP**: A healthy patient incorrectly diagnosed with the disease.
  - **TN**: A healthy patient correctly diagnosed as healthy.
  - **FN**: A patient with the disease incorrectly diagnosed as healthy.

Section 2: Accuracy, Precision, Recall, F1 Score

Accuracy, Precision, Recall, F1 Score are the main metrics used to evaluate a classification model, just like MSE/MAE/RMSE/R2 for regression models. They are calculated from the True/False positive/negative rates.

Accuracy

- **Definition**: The proportion of correctly classified instances (both positives and negatives) out of the total instances.
- **Limitation**: Accuracy can be misleading in imbalanced datasets (where one class is much more frequent than the other). For example, in a dataset where 95% of the

samples belong to one class, a model that always predicts the majority class would have 95% accuracy but would be ineffective.

Precision

- **Definition**: The proportion of correctly predicted positive instances out of all instances predicted as positive. Evaluates FALSE POSITIVES
- **Limitation**: Precision does not consider false negatives, so it might not fully capture the model's performance when false negatives are important.

Recall (also known as Sensitivity or True Positive Rate)

- **Definition**: The proportion of correctly predicted positive instances out of all actual positives. Evaluates FALSE NEGATIVES
- **Limitation**: Recall does not consider false positives, which can be critical in scenarios where incorrect positive predictions have severe consequences.

F1 Score

- **Definition**: The average of precision and recall, providing a balance between the two metrics.
- **Limitation**: The F1 score assumes that precision and recall are equally important, which might not always be the case.

Section 3: ROC and AUC

ROC Curve (Receiver Operating Characteristic Curve):

Definition: A plot that illustrates the performance of a binary classifier system as its discrimination threshold is varied. The curve plots the True Positive Rate (Recall) against the False Positive Rate (1 - Specificity).

Breakdown:

- **Thresholds:** The ROC curve is constructed by varying the decision threshold that converts model probabilities into class labels. For instance, you might start with a threshold of 0.1, meaning any probability above 0.1 is classified as positive, and calculate the True Positive Rate (TPR) and False Positive Rate (FPR) at this threshold.
- **Continuous Curve:** By repeating this process for various thresholds (e.g., 0.2, 0.3, ..., 0.9), you obtain different TPR and FPR values. Plotting these points on the ROC curve gives you a comprehensive view of the model's performance across different decision thresholds.

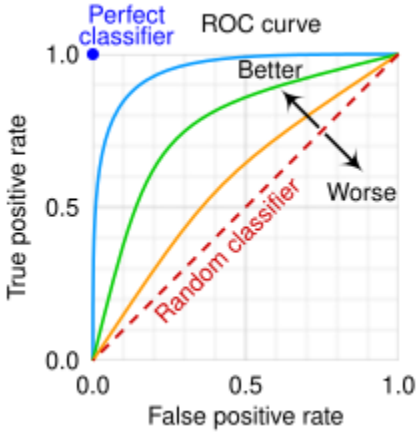
- **Interpretability:** A model with a curve closer to the top-left corner (high TPR and low FPR) indicates better performance, i.e. instances with higher predicted probabilities of being positive are indeed more likely to be positive.

AUC (Area Under the Curve):

- **Definition**: The area under the ROC curve, representing the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance.
- **Interpretation**:
    - AUC = 1 indicates a perfect model.
    - AUC = 0.5 indicates a model no better than random guessing.
    - AUC > 0.7 is typically considered acceptable, while AUC > 0.9 is considered excellent.

Summary:

- **ROC Curve** uses model probabilities to plot the trade-off between True Positive Rate and False Positive Rate at various thresholds, providing a detailed view of model performance across different classification thresholds.
- **AUC** quantifies the overall performance of the model in a single number by measuring the area beneath the ROC curve. Both ROC and AUC are valuable for assessing the effectiveness of a classification model's probabilistic predictions.
- **Limitations:** ROC/AUC can sometimes be misleading when dealing with imbalanced datasets.



Section 4: Brier Scores

- **Definition**: The Brier score measures the mean squared difference between predicted probabilities and the actual binary outcomes. Essentially, the "residual".
- **Interpretation**:

    - A Brier score ranges between 0 and 1, where 0 represents a perfect model and 1 represents the worst possible prediction.
    - Similar to AUC, the Brier score takes into account the calibration of the predicted probabilities, making it sensitive to both the accuracy of the classification and the confidence of the model's predictions.
- **Limitations:** Might not be really relevant to your problem if you don't care about probabilities and only the predicted labels/categories.

Chapter 4: Typical Supervised Problem Workflow

(This is mostly the same as from the last study guide)

- Data Preprocessing
    - Imputation (null values)
    - Encoding (categorical values -> numeric)
    - Scaling of Numeric Features (StandardScaler or MinMaxScaler)
    - Feature creation (do you want to create or transform columns? Like add them together, do binning, etc)
- Correlation Analysis
    - Are we afflicted by multi-collinearity?
    - Which features have high linear correlations with the target?
    - NOTE: Just because a feature has a low correlation doesn't mean that it's useless. It could still have non-linear predictive power that a tree-based model can capture
    - Correlations also have less of an impact on CLASSIFICATION models
- Feature Selection
    - Which columns will you use to train your model?
    - This is typically an iterative process
    - You might want to drop columns with high multi-collinearity or that have low correlations
    - On a first pass, might just want to keep everything and see how it goes #yolo
    - This could also be the step for PCA/Dimensionality Reduction if you so choose
- Over/Under Sample as necessary for IMBALANCED classification problems
- Create your Train/Test splits
    - i.e. 80/20, 75/25, 70/30
    - STRATIFY if needed for Classification

This is all data cleaning/engineering 😿

- Begin your ML Experiment (an experiment is a collection of multiple algorithms and metrics used for tracking and reproducibility)
    - Train an assortment of algorithms (linear and tree-based, KNN, SVM)

        - Generate Training and Testing Metrics & Test Set Visualizations
        - Check for Overfitting for comparing the Training & Testing Metrics
        - Keep Track of the Test set metrics & visualizations for model comparisons
        - Compare all test set metrics and visualizations to find the "best" model
        - Use the visualizations to identify the model limitations
        - For the best model, try and look at Feature Importances
            - For features with low importance, it might be worth going back and dropping them in the Feature Selection step, then re-training the model
            - Fewer Columns = Less Complexity = More Explainability = More Computational Efficiency
            - Goal is Most Predictable with Least Complexity
            - Make sure you understand WHY the model is making a prediction, to the best of your ability
            - Do the feature importances make sense given your domain knowledge?

- Tune best model hyperparameters as needed (optional)
- Retrain best model on ALL training data (not just the train/test split) (will do later)
- Save the model for future use in deployed production systems or ML applications (will do later)

This workflow is the same for both Regression & Classification.

Chapter 5: Vocabulary

- Classification vs Regression
- Support Vector Machines
- Hyperplanes
- K-Nearest Neighbors
- Distance Metrics: Euclidean, Manhattan, Cosine..
- Type 1/Type 2 Errors
- True/False Positives/Negatives
- Confusion Matrix
- Accuracy
- Precision
- Recall (also known as Sensitivity)
- ROC Curve
- AUC
- Brier Score
- Regularization (L1, L2)
- Logistic Regression
- Sigmoid Function
- S-Curve
- Imbalanced Classification

- Bonus: Over/Under Sampling
- Bonus: SMOTE
- Model Evaluation
- Model Selection
- Feature Importances
- Train/Test split (STRATIFY)
- Model Explainability
- Model Experiment Tracking
- Bonus: Hyperparameter tuning, Cross-Validation, Random_State
- Bonus: MLOps, Model Deployment, MLFlow